Problem 4: PCA from Scratch vs. UMAP on Cancer Gene Expression

This notebook implements every part of Problem 4 using the local `gene+expression+cancer+rna+seq.zip` archive. PCA is implemented from scratch in NumPy using both SVD and the Gram-matrix eigendecomposition trick. UMAP uses the `umap-learn` package, as allowed by the prompt.

In [12]:
```
from pathlib import Path
import io
import tarfile
import zipfile
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split

plt.style.use("seaborn-v0_8-whitegrid")
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
```

4(a) Look at the data

In [13]:
```
def read_gene_expression_archive(zip_path="gene+expression+cancer+rna+seq.zip"):
    """Load data.csv and labels.csv from the UCI zip without manual extraction."""
    zip_path = Path(zip_path)
    extracted_dir = Path("TCGA-PANCAN-HiSeq-801x20531")
    extracted_data = extracted_dir / "data.csv"
    extracted_labels = extracted_dir / "labels.csv"

    if extracted_data.exists() and extracted_labels.exists():
        return pd.read_csv(extracted_data), pd.read_csv(extracted_labels)

    with zipfile.ZipFile(zip_path) as zf:
        tar_name = next(name for name in zf.namelist() if name.endswith(".tar.gz"))
        tar_bytes = io.BytesIO(zf.read(tar_name))

    with tarfile.open(fileobj=tar_bytes, mode="r:gz") as tf:
        members = {member.name: member for member in tf.getmembers()}
        data_name = next(name for name in members if name.endswith("/data.csv"))
        labels_name = next(name for name in members if name.endswith("/labels.csv"))

        with tf.extractfile(members[data_name]) as data_file:
```

```
            data = pd.read_csv(data_file)
        with tf.extractfile(members[labels_name]) as labels_file:
            labels = pd.read_csv(labels_file)

    return data, labels


data_raw, labels_raw = read_gene_expression_archive()
print("data.csv shape:", data_raw.shape)
print("labels.csv shape:", labels_raw.shape)
print("data columns, first 5:", list(data_raw.columns[:5]))
print("labels columns:", list(labels_raw.columns))

gene_id_col = data_raw.columns[0]
label_id_col = labels_raw.columns[0]
label_col = labels_raw.columns[1]

X_df = data_raw.drop(columns=[gene_id_col])
y = labels_raw[label_col].astype(str)

print("X shape after dropping ID:", X_df.shape)
print("Class counts:")
display(y.value_counts().sort_index())
```

```
data.csv shape: (801, 20532)
labels.csv shape: (801, 2)
data columns, first 5: ['Unnamed: 0', 'gene_0', 'gene_1', 'gene_2', 'gene_3']
labels columns: ['Unnamed: 0', 'Class']
X shape after dropping ID: (801, 20531)
Class counts:
Class
BRCA    300
COAD     78
KIRC    146
LUAD    141
PRAD    136
Name: count, dtype: int64
```

In [14]:
```
assert X_df.shape == (801, 20531), f"Expected (801, 20531), got {X_df.shape}"
assert len(y) == len(X_df), "Labels and data must have the same number of rows."
```

In [15]:
```
# Full describe is very wide, so also show a reproducible subset of gene columns.
random_gene_cols = X_df.sample(n=8, axis=1, random_state=RANDOM_STATE).columns
print("Summary for a few random gene columns:")
display(X_df[random_gene_cols].describe())

value_min = X_df.to_numpy().min()
value_max = X_df.to_numpy().max()
print(f"Approximate expression-value range in this dataset: {value_min:.3f} to {val
```

```
Summary for a few random gene columns:
```

	gene_1272	gene_3222	gene_6423	gene_16755	gene_12457	gene_6956	gene_5921
count	801.000000	801.000000	801.000000	801.000000	801.000000	801.000000	801.000000
mean	11.063668	6.614294	10.556457	0.278427	0.010162	7.995829	6.716145
std	1.051581	2.938234	0.549155	0.564358	0.092842	1.192065	0.457912
min	5.064111	0.000000	8.978241	0.000000	0.000000	3.006711	4.689584
25%	10.505047	4.502872	10.170013	0.000000	0.000000	7.343052	6.429167
50%	11.172234	6.300005	10.500324	0.000000	0.000000	8.064150	6.724732
75%	11.677772	8.917253	10.899795	0.416083	0.000000	8.743064	6.985762
max	14.292135	14.245501	12.795739	3.828093	1.725305	11.635110	8.642691

```
Approximate expression-value range in this dataset: 0.000 to 20.779
```

The expression values are already on a normalized log-like scale, so I do not apply an additional log transform. I still standardize gene columns before PCA/UMAP so each gene contributes comparably; otherwise high-variance genes would dominate Euclidean distances and the PCA objective.

In [16]:
```
gene_means = X_df.mean(axis=0)
gene_stds = X_df.std(axis=0, ddof=0).replace(0, 1.0)
X_std_df = (X_df - gene_means) / gene_stds
X_std = X_std_df.to_numpy(dtype=np.float64, copy=False)

print("Standardized mean range:", X_std_df.mean(axis=0).min(), "to", X_std_df.mean(
print("Standardized std range:", X_std_df.std(axis=0, ddof=0).min(), "to", X_std_df
```

```
Standardized mean range: -7.746335131117447e-15 to 7.826171393562402e-15
Standardized std range: 0.0 to 1.0000000000000007
```

4(b) PCA via SVD

In [17]:
```
def pca_svd(X, k):
    """
    PCA from scratch using the SVD of centered data.

    Returns:
        components: shape (k, d), rows are principal axes
        projected: shape (N, k), centered data projected onto components
        explained_variance: shape (k,), sample variance explained by each component
    """
    X = np.asarray(X, dtype=np.float64)
    n_samples = X.shape[0]
    if not 1 <= k <= min(X.shape):
        raise ValueError("k must be between 1 and min(N, d).")

    X_centered = X - X.mean(axis=0)
    U, singular_values, Vt = np.linalg.svd(X_centered, full_matrices=False)
```

```
    components = Vt[:k]
    projected = X_centered @ components.T
    explained_variance = (singular_values[:k] ** 2) / (n_samples - 1)
    return components, projected, explained_variance


components_svd_2, pca2_svd, explained_svd_2 = pca_svd(X_std, k=2)
print("Explained variances for PC1 and PC2:", explained_svd_2)
```

```
Explained variances for PC1 and PC2: [2138.4510195  1776.17511467]
```

4(c) PCA via covariance eigendecomposition

The direct covariance matrix would be `20531 x 20531`, or about 3.4 GB as float64 values before any eigendecomposition overhead. Since `N << d`, the Gram matrix `Xc Xc.T / (N - 1)` is only `801 x 801`, and its nonzero eigenvalues match the nonzero covariance eigenvalues.

In [18]:
```
def pca_eig(X, k):
    """
    PCA from scratch using the small Gram matrix eigendecomposition.

    This avoids materializing the d x d covariance matrix when d is very large.
    """
    X = np.asarray(X, dtype=np.float64)
    n_samples = X.shape[0]
    if not 1 <= k <= min(X.shape):
        raise ValueError("k must be between 1 and min(N, d).")

    X_centered = X - X.mean(axis=0)
    gram = (X_centered @ X_centered.T) / (n_samples - 1)

    eigenvalues, U = np.linalg.eigh(gram)
    order = np.argsort(eigenvalues)[::-1]
    eigenvalues = eigenvalues[order]
    U = U[:, order]

    components = []
    for j in range(k):
        eigenvalue = max(eigenvalues[j], 0.0)
        if eigenvalue <= 1e-12:
            v = np.zeros(X.shape[1])
        else:
            v = (X_centered.T @ U[:, j]) / np.sqrt((n_samples - 1) * eigenvalue)
            v = v / np.linalg.norm(v)
        components.append(v)

    components = np.vstack(components)
    projected = X_centered @ components.T
    explained_variance = eigenvalues[:k]
    return components, projected, explained_variance
```

```
components_eig_2, pca2_eig, explained_eig_2 = pca_eig(X_std, k=2)
print("SVD explained variances:", explained_svd_2)
print("Eigen explained variances:", explained_eig_2)
print("Variances agree:", np.allclose(explained_svd_2, explained_eig_2, rtol=1e-8,

axis_abs_dot = np.abs(np.sum(components_svd_2 * components_eig_2, axis=1))
print("Absolute dot product between matching axes:", axis_abs_dot)
print("Axes agree up to sign:", np.allclose(axis_abs_dot, np.ones_like(axis_abs_dot
```

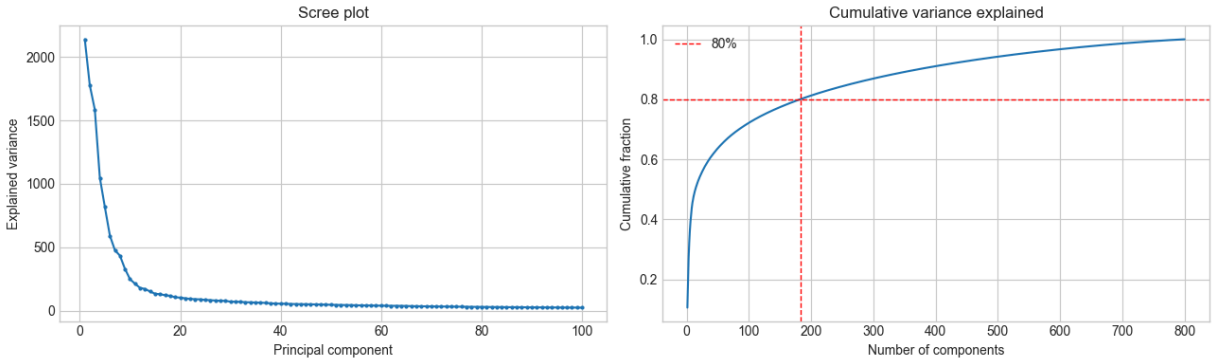
```
SVD explained variances: [2138.4510195  1776.17511467]
Eigen explained variances: [2138.4510195  1776.17511467]
Variances agree: True
Absolute dot product between matching axes: [1. 1.]
Axes agree up to sign: True
```

The explained variances should match up to floating-point error, and the absolute dot products should be essentially 1.0. The absolute value is necessary because eigenvectors are defined only up to sign.

4(d) Scree plot

In [19]:
```
max_components = min(X_std.shape) - 1
components_all, projected_all, explained_all = pca_svd(X_std, k=max_components)

variance_fraction = explained_all / explained_all.sum()
cumulative_fraction = np.cumsum(variance_fraction)
components_for_80 = int(np.searchsorted(cumulative_fraction, 0.80) + 1)

print(f"Computed PCA components: {max_components}")
print(f"Components needed for 80% variance: {components_for_80}")
print(f"Original number of genes: {X_std.shape[1]}")

plot_k = min(100, len(explained_all))
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
axes[0].plot(np.arange(1, plot_k + 1), explained_all[:plot_k], marker="o", markersi
axes[0].set_title("Scree plot")
axes[0].set_xlabel("Principal component")
axes[0].set_ylabel("Explained variance")

axes[1].plot(np.arange(1, len(cumulative_fraction) + 1), cumulative_fraction)
axes[1].axhline(0.80, color="red", linestyle="--", linewidth=1, label="80%")
axes[1].axvline(components_for_80, color="red", linestyle="--", linewidth=1)
axes[1].set_title("Cumulative variance explained")
axes[1].set_xlabel("Number of components")
axes[1].set_ylabel("Cumulative fraction")
axes[1].legend()
plt.tight_layout()
```

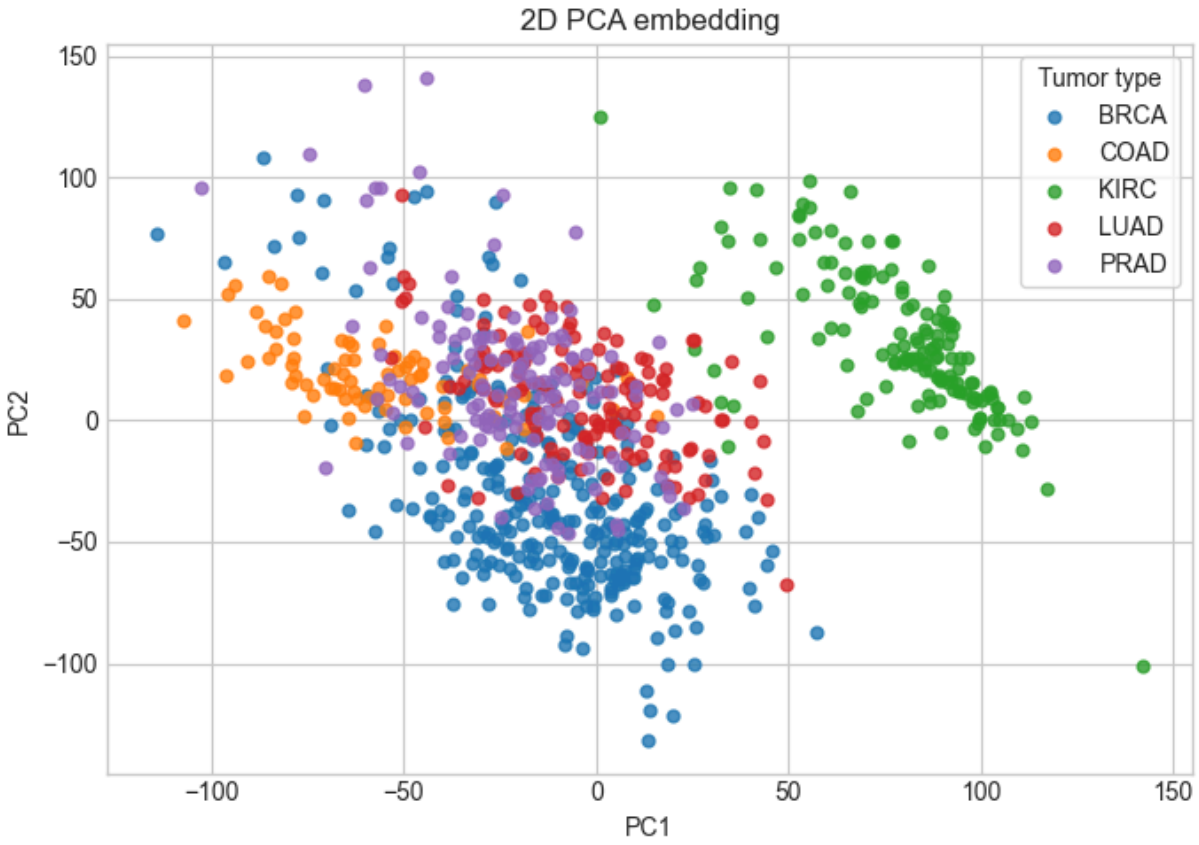
```
Computed PCA components: 800
Components needed for 80% variance: 183
Original number of genes: 20531
```



The number printed above is far smaller than the original 20,531 genes, showing that the dominant variation in these samples lives in a much lower-dimensional subspace.

4(e) 2D PCA visualization

In [20]:
```
def scatter_embedding(embedding, labels, title, ax=None, xlabel="Dimension 1", ylab
    if ax is None:
        fig, ax = plt.subplots(figsize=(7, 5))
    classes = sorted(pd.Series(labels).unique())
    cmap = plt.get_cmap("tab10")
    for i, cls in enumerate(classes):
        mask = np.asarray(labels) == cls
        ax.scatter(embedding[mask, 0], embedding[mask, 1], s=24, alpha=0.8, color=c
    ax.set_title(title)
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.legend(title="Tumor type", frameon=True)
    return ax


pca_2d = projected_all[:, :2]
fig, ax = plt.subplots(figsize=(7, 5))
scatter_embedding(pca_2d, y.to_numpy(), "2D PCA embedding", ax=ax, xlabel="PC1", yl
plt.tight_layout()
```



PCA should show meaningful tumor-type structure because tissue-specific expression patterns explain a large share of the variance. In a typical run on this dataset, several classes form clear regions, while the two adenocarcinoma classes can be closer or more partially mixed in a purely linear 2D projection.

4(f) What do PC1 and PC2 mean?

In [21]:
```
gene_names = np.asarray(X_df.columns)


def top_loadings(component, gene_names, n=10):
    idx = np.argsort(np.abs(component))[::-1][:n]
    return pd.DataFrame({
        "gene_index": idx,
        "gene": gene_names[idx],
        "loading": component[idx],
        "abs_loading": np.abs(component[idx]),
    })

pc1_top = top_loadings(components_all[0], gene_names, n=10)
pc2_top = top_loadings(components_all[1], gene_names, n=10)

print("Top absolute loadings for PC1:")
display(pc1_top)
print("Top absolute loadings for PC2:")
display(pc2_top)
```

```
pc1_genes = set(pc1_top["gene"])
pc2_genes = set(pc2_top["gene"])
print("Overlap among top-10 genes:", sorted(pc1_genes & pc2_genes))
```

```
Top absolute loadings for PC1:
```

	gene_index	gene	loading	abs_loading
0	19862	gene_19862	0.019002	0.019002
1	17360	gene_17360	0.018985	0.018985
2	13489	gene_13489	0.018966	0.018966
3	15158	gene_15158	0.018777	0.018777
4	7031	gene_7031	0.018740	0.018740
5	7019	gene_7019	0.018657	0.018657
6	10788	gene_10788	0.018629	0.018629
7	13507	gene_13507	-0.018624	0.018624
8	6543	gene_6543	0.018595	0.018595
9	2288	gene_2288	-0.018592	0.018592

```
Top absolute loadings for PC2:
```

	gene_index	gene	loading	abs_loading
0	3612	gene_3612	-0.020273	0.020273
1	1222	gene_1222	0.019863	0.019863
2	14699	gene_14699	-0.019862	0.019862
3	1010	gene_1010	0.019829	0.019829
4	19498	gene_19498	0.019505	0.019505
5	1113	gene_1113	-0.019409	0.019409
6	3603	gene_3603	-0.019346	0.019346
7	364	gene_364	-0.019225	0.019225
8	11153	gene_11153	0.019222	0.019222
9	19866	gene_19866	0.019133	0.019133

```
Overlap among top-10 genes: []
```

The top-loading genes are the genes that drive each principal direction most strongly. PC1 and PC2 usually have different top genes, which means they summarize different expression programs.

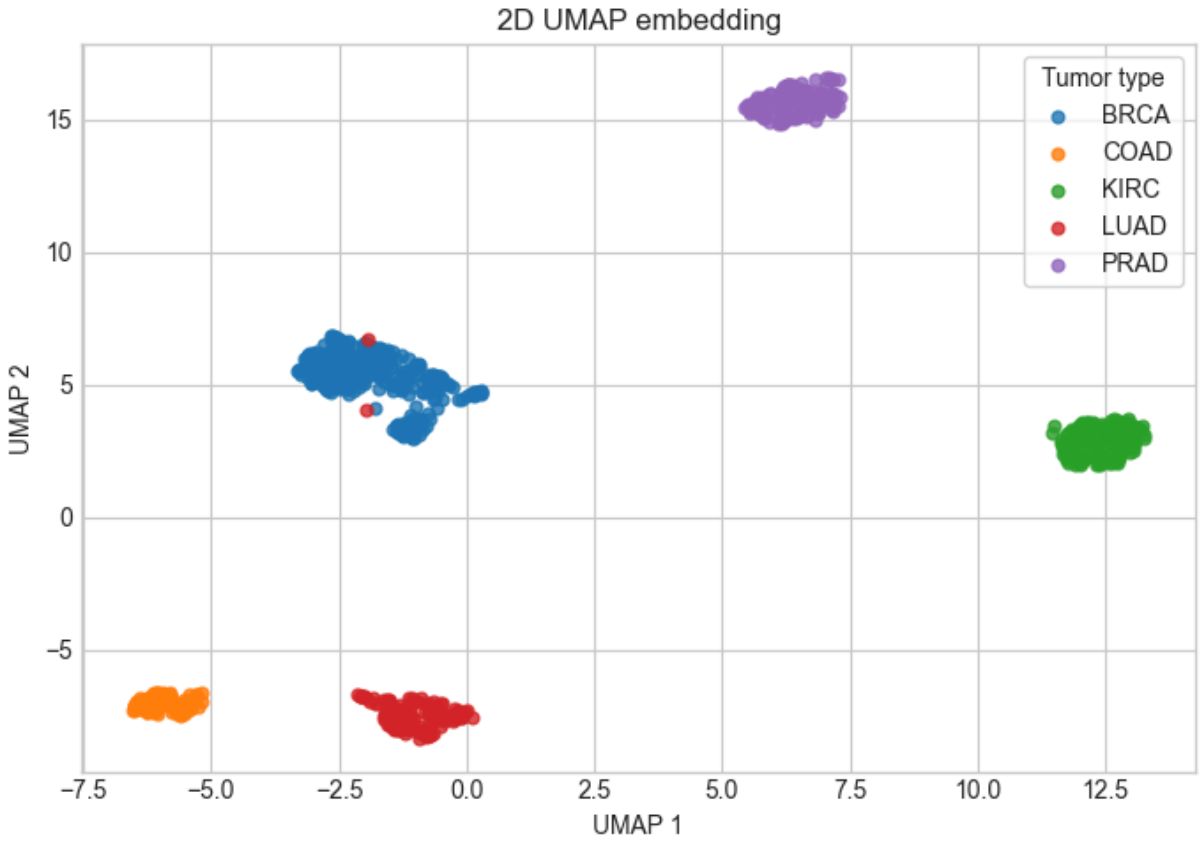
4(g) UMAP via library

If the next import fails, run this in a notebook cell first:

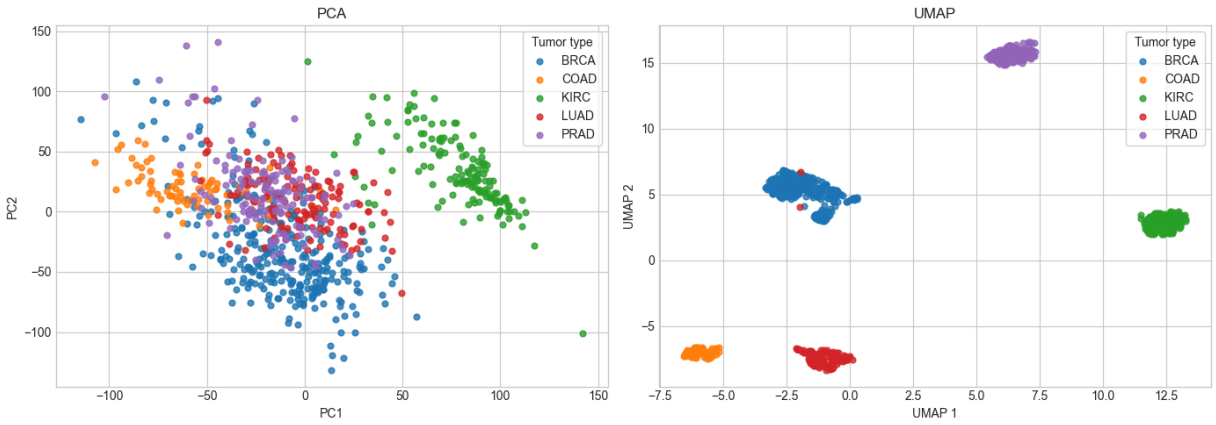
```
%pip install umap-learn
```

In [22]:
```
try:
    import umap.umap_ as umap
except ImportError as exc:
    raise ImportError("Install UMAP with `%pip install umap-learn`, then rerun this

umap_model = umap.UMAP(n_components=2, random_state=RANDOM_STATE)
umap_2d = umap_model.fit_transform(X_std)

fig, ax = plt.subplots(figsize=(7, 5))
scatter_embedding(umap_2d, y.to_numpy(), "2D UMAP embedding", ax=ax, xlabel="UMAP 1
plt.tight_layout()
```

```
C:\Users\HarryMyers\AppData\Roaming\Python\Python314\site-packages\tqdm\auto.py:21:
TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See https://
ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
C:\Users\HarryMyers\AppData\Roaming\Python\Python314\site-packages\umap\umap_.py:195
2: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(
```



In [23]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
scatter_embedding(pca_2d, y.to_numpy(), "PCA", ax=axes[0], xlabel="PC1", ylabel="PC
scatter_embedding(umap_2d, y.to_numpy(), "UMAP", ax=axes[1], xlabel="UMAP 1", ylabe
plt.tight_layout()
```

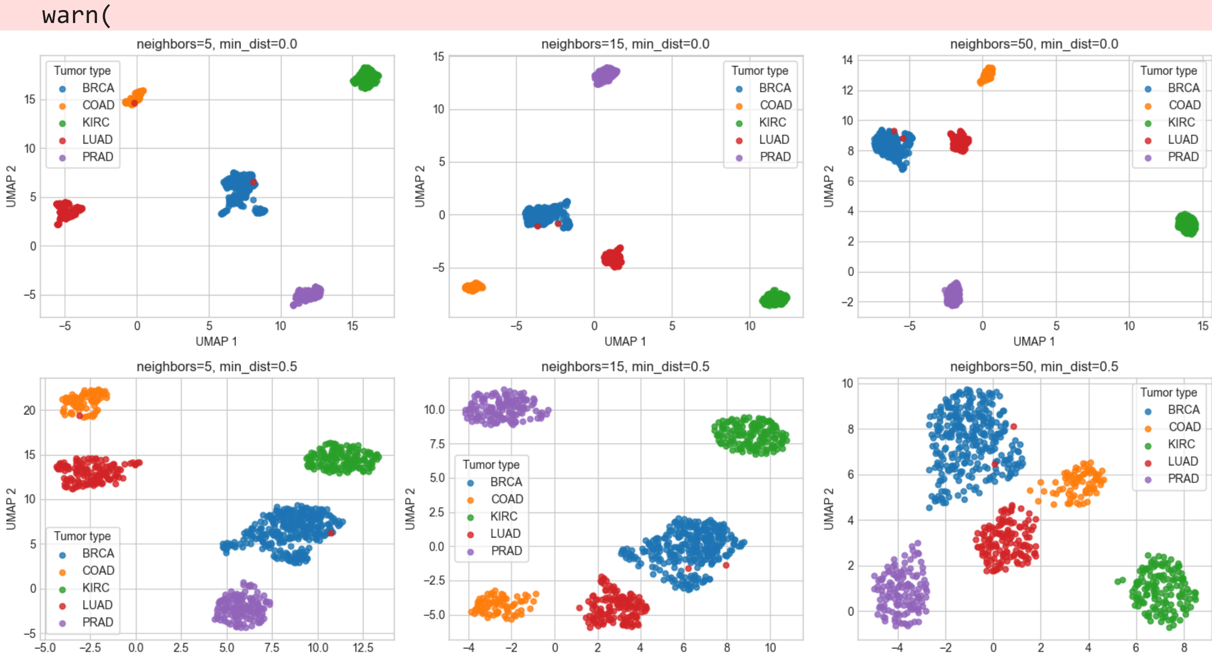


UMAP typically gives more visually separated groups than PCA because it can preserve nonlinear local neighborhoods instead of being restricted to one global linear projection. The tradeoff is that UMAP distances and axes are less directly interpretable than PCA loadings.

4(h) UMAP hyperparameter exploration

In [24]:
```
neighbor_values = [5, 15, 50]
min_dist_values = [0.0, 0.5]

fig, axes = plt.subplots(len(min_dist_values), len(neighbor_values), figsize=(15, 8
umap_grid_embeddings = {}

for row, min_dist in enumerate(min_dist_values):
    for col, n_neighbors in enumerate(neighbor_values):
        reducer = umap.UMAP(
            n_components=2,
            n_neighbors=n_neighbors,
            min_dist=min_dist,
            random_state=RANDOM_STATE,
        )
        emb = reducer.fit_transform(X_std)
        umap_grid_embeddings[(n_neighbors, min_dist)] = emb
        scatter_embedding(
            emb,
            y.to_numpy(),
            f"neighbors={n_neighbors}, min_dist={min_dist}",
            ax=axes[row, col],
            xlabel="UMAP 1",
            ylabel="UMAP 2",
        )

plt.tight_layout()
```

```
C:\Users\HarryMyers\AppData\Roaming\Python\Python314\site-packages\umap\umap_.py:195
2: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(
C:\Users\HarryMyers\AppData\Roaming\Python\Python314\site-packages\umap\umap_.py:195
2: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(
C:\Users\HarryMyers\AppData\Roaming\Python\Python314\site-packages\umap\umap_.py:195
2: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(
C:\Users\HarryMyers\AppData\Roaming\Python\Python314\site-packages\umap\umap_.py:195
2: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(
C:\Users\HarryMyers\AppData\Roaming\Python\Python314\site-packages\umap\umap_.py:195
2: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(
C:\Users\HarryMyers\AppData\Roaming\Python\Python314\site-packages\umap\umap_.py:195
2: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(
```



Smaller `n_neighbors` emphasizes very local structure, so clusters usually become tighter and more fragmented. Larger `n_neighbors` preserves more global structure, while larger `min_dist` spreads points out inside each group. For a lab-facing visualization, I would usually ship a middle or larger-neighborhood setting, such as `n_neighbors=15` or `50`, with `min_dist=0.0` if the goal is clean class separation.

4(i) Downstream task: does the embedding actually help?

In [25]:
```
indices = np.arange(len(y))
train_idx, test_idx = train_test_split(
    indices,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y,
)

X_train_raw = X_df.iloc[train_idx].to_numpy(dtype=np.float64, copy=True)
X_test_raw = X_df.iloc[test_idx].to_numpy(dtype=np.float64, copy=True)
y_train = y.iloc[train_idx].to_numpy()
y_test = y.iloc[test_idx].to_numpy()

train_means = X_train_raw.mean(axis=0)
train_stds = X_train_raw.std(axis=0, ddof=0)
train_stds[train_stds == 0] = 1.0
X_train_full = (X_train_raw - train_means) / train_stds
X_test_full = (X_test_raw - train_means) / train_stds

print("Train size:", X_train_full.shape[0], "Test size:", X_test_full.shape[0])
print("Train class counts:")
display(pd.Series(y_train).value_counts().sort_index())
print("Test class counts:")
display(pd.Series(y_test).value_counts().sort_index())
```

```
Train size: 640 Test size: 161
Train class counts:
BRCA    240
COAD     62
KIRC    116
LUAD    113
PRAD    109
Name: count, dtype: int64
Test class counts:
BRCA    60
COAD    16
KIRC    30
LUAD    28
PRAD    27
Name: count, dtype: int64
```

In [26]:
```
def fit_logistic_and_score(X_train, X_test, y_train, y_test):
    clf = LogisticRegression(max_iter=2000, random_state=RANDOM_STATE)
    clf.fit(X_train, y_train)
    pred = clf.predict(X_test)
    return accuracy_score(y_test, pred), clf

# Full 20,531-gene baseline.
full_acc, full_clf = fit_logistic_and_score(X_train_full, X_test_full, y_train, y_t

# PCA embedding fit on training data only, then applied to test data.
# Fit 10 components once so the next cell can also answer the reflection question.
pca_train_components_10, X_train_pca_10d, _ = pca_svd(X_train_full, k=10)
pca_train_center = X_train_full.mean(axis=0)
X_test_pca_10d = (X_test_full - pca_train_center) @ pca_train_components_10.T
X_train_pca_2d = X_train_pca_10d[:, :2]
```

```
X_test_pca_2d = X_test_pca_10d[:, :2]
pca_acc, pca_clf = fit_logistic_and_score(X_train_pca_2d, X_test_pca_2d, y_train, y

# UMAP fit on training data only, then transformed for test data.
umap_train_model = umap.UMAP(n_components=2, random_state=RANDOM_STATE)
X_train_umap_2d = umap_train_model.fit_transform(X_train_full)
X_test_umap_2d = umap_train_model.transform(X_test_full)
umap_acc, umap_clf = fit_logistic_and_score(X_train_umap_2d, X_test_umap_2d, y_trai

accuracy_table = pd.DataFrame({
    "method": ["Full gene expression", "PCA", "UMAP"],
    "dimensions": [X_train_full.shape[1], 2, 2],
    "test_accuracy": [full_acc, pca_acc, umap_acc],
})
display(accuracy_table)
```

```
C:\Users\HarryMyers\AppData\Roaming\Python\Python314\site-packages\umap\umap_.py:195
2: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(
```

	method	dimensions	test_accuracy
0	Full gene expression	20531	0.981366
1	PCA	2	0.633540
2	UMAP	2	0.981366

In [27]:
```
# Optional: check whether 10 PCA components help relative to 2D PCA.
pca10_acc, pca10_clf = fit_logistic_and_score(X_train_pca_10d, X_test_pca_10d, y_tr
print(f"10D PCA logistic-regression test accuracy: {pca10_acc:.4f}")
```

```
10D PCA logistic-regression test accuracy: 0.9938
```

The full-gene classifier is the strongest baseline because it keeps all measured information. A 2D embedding is useful for visualization, but it is an aggressive compression; 10 PCA components often recover more predictive signal than 2 PCA components while still being far smaller than 20,531 genes.

4(j) Summary

In [28]:
```
summary_table = accuracy_table.copy()
summary_table.loc[len(summary_table)] = ["PCA", 10, pca10_acc]
display(summary_table.sort_values(["dimensions", "method"]))
```

	method	dimensions	test_accuracy
1	PCA	2	0.633540
2	UMAP	2	0.981366
3	PCA	10	0.993789
0	Full gene expression	20531	0.981366

Recommendation: use UMAP or 2D PCA for exploratory visualization, but use the full standardized expression vector, or a higher-dimensional PCA representation, for downstream prediction. PCA is best when interpretability matters because loadings identify influential genes; UMAP is best when the lab wants a visually clean map of local tumor neighborhoods. I would not treat the 2D plots alone as proof of diagnostic performance, because the classifier accuracy table is the more direct test.